Entrega E — Matriz de experimentos (Hora 6) — versão preliminar

Regras: **uma falha de cada vez**, previsão escrita **antes** de rodar, sem carga agressiva (no máximo 20 requisições sequenciais), recuperar e confirmar o retorno após cada uma. Ordem do enunciado: E1 parar A → E2 endpoint lento → E3 parar watchdog → E4 erro `/erro`. E5–E7 são nossos, motivados pelos achados (`docs/achados.md`).

Legenda de status: ✔ executado com evidência · ◆ **parcial** executado parcialmente (Horas 3–4) · previsto previsto (05–12/11)

#	Hipótese	Procedimento	Resultado esperado	Evidência	Conclusão	Status
E1	Parar A não derruba o serviço; 1–3 requisições pagam ~1 s, porque o Nginx só retira A após 3 falhas e cada uma espera `proxy_connect_timeout 1s`	`docker compose stop rust-a`; `scripts/teste_sintetico.sh 20`; esperar 28 s; ver watchdog e `/alerts`; `start rust-a`; 20 req	20/20 com 200 via B; 1–3 req ~1 s; watchdog DOWN em ~15 s; `NoDown` firing em ~20–25 s; após start, A volta	`evidencias/preliminar/05_no_parado_e_recuperacao.txt`, `05b`, `05c`, `11`	**Confirmada**, com ressalvas: (a) parte das falhas foi absorvida pelo teste sintético (achado 3); (b) o tráfego leva até ~10 s para voltar a A depois do start (achado 4); (c) em `05` nenhuma requisição foi lenta porque A já estava fora do pool (achado 8)	◆ **parcial**
E2	Com A lento (2,5 s), o watchdog marca A DOWN (sonda tem timeout 2 s), mas o Nginx continua mandando tráfego a A (read timeout 3 s): ~metade das requisições leva ~2,5 s, sem erro. Watchdog e Nginx discordam	`scripts/admin.sh atraso rust-a 2500`; 20 req; ver p95 no Prometheus e log do watchdog; `atraso rust-a 0`	20/20 com 200; ~10 com ~2,5 s; watchdog DOWN; `LatenciaP95Alta` após 1 min; **sem** retry no Nginx	—	—	previsto
E2b	Com atraso de 3,5 s (> read timeout), o Nginx estoura o timeout e refaz em B: o usuário espera ~3,5 s (3 s + B) e B recebe o dobro — retry amplifica carga	`atraso rust-a 3500`; 10 req; contar `upstream_status=504, 200`	10/10 com 200; metade com ~3 s; log com 2 upstreams	—	—	previsto
E3	Parar o watchdog não afeta o usuário, mas `MonitorParado` dispara em ~35 s; nesse intervalo uma queda real de nó **não** gera `NoDown`	`docker compose stop watchdog`; consultar `/alerts` a cada 10 s; (opcional) `stop rust-a` junto	Usuário 200; `ColetaFalhando{job=watchdog}` ~15 s; `MonitorParado` ~30–35 s; `NoDown` ausente mesmo com A parado	—	—	previsto

#	Hipótese	Procedimento	Resultado esperado	Evidência	Conclusão	Status
E4	Erro da aplicação (500) **não** é refeito pelo Nginx (500 fora de `proxy_next_upstream`) e a taxa de erro passa de 2%	20× `curl localhost:8080/erro` intercalado com 20× `/`	20 respostas 500 ao usuário, cada uma de **um** só upstream; `TaxaErro5xxAlta` firing após ~30 s	—	—	previsto
E5	Nó não pronto (readiness 503) é retirado do balanceamento **sem reinício**; live segue 200	`admin.sh ready rust-a 0`; live/check; 10 req; esperar DOWN; `ready rust-a 1`	live 200 / ready 503; 10/10 com 200 via B; log `upstream_status=503, 200`; watchdog DOWN em 15 s; RestartCount inalterado	`03_health_checks.txt`	**Confirmada** (Hora 3). Efeito colateral: falso positivo no SLI de erro (achado 5, corrigido)	◆ **parcial**
E6	O tempo de retorno do tráfego após `start` é governado pelo `resolver valid=10s` e cai para ~5 s com `valid=5s`	`scripts/medir_retorno.sh rust-a` com `valid=10s` (base) e com `valid=5s`; 3 rodadas cada	Base ~10 s; `valid=5s` ~5 s; watchdog ~2 s nos dois	`11_tempo_retorno_trafego.txt` (base, 2 rodadas)	Parcial: base medida	◆ **parcial**
E7	Config do Nginx quebrada é barrada por `nginx -t` antes de afetar o serviço	Três cópias quebradas (sem `;`, host inexistente, upstream inexistente) + `nginx -t`	Erro com arquivo e linha; Nginx em produção intacto	`06_config_quebrada.txt`	Ver arquivo (resultado comentado no guia de estudo)	✔

Como cada experimento vai ser registrado (modelo)

```
=== E<n> — <título> — <data/hora UTC> — operador: <nome> ===
PREVISÃO (escrita antes): ...
$ comando 1
<saída>
OBSERVADO: ...
RECUPERAÇÃO: comando + prova (20 req balanceadas, alertas resolvidos, RestartCount)
CONCLUSÃO: confirmada | refutada | parcialmente — porque ...
```

Script de apoio a ser criado na Hora 6: `scripts/experimentos.sh E2` etc., no mesmo estilo de `scripts/evidencias_preliminar.sh`, gravando em `evidencias/experimentos/`.